

OTH Regensburg

Studiengang: Technische Informatik

Projektdokumentation

HydroNodeStation

Fach: Ausgewählte Projekte der IT

Semester: IT6

Name: Felix Knoll

Abgabedatum: 7. September 2026

Inhaltsverzeichnis

1.	Einleitung und Projektbeschreibung	3
2.	Hardware	3
3.	Firmware und Software	6
4.	Inbetriebnahme	8
5.	Energieeffizienz	11
6.	Tests und Validierung	12
7.	HydroNodeNetzwerk	13
8.	Open-Source-Veröffentlichung	14
9.	Stundenaufwand	14
10.	KI-Verwendung	14
11.	Abweichungen, Probleme und Lösungen	15
12.	Fazit	17
A.	Stundenaufwand: Detailtabellen	18

1. Einleitung und Projektbeschreibung

HydroNodeStation ist eine energieeffiziente, solar-betriebene Umweltmessstation für den autonomen Außeneinsatz ohne externe Infrastruktur. Sie überträgt meteorologische und luftqualitätsrelevante Messdaten über LoRaWAN an das Helium-Netzwerk und von dort in Echtzeit ins HydroNode-Backend sowie an ein öffentliches Web-Dashboard. Kernziele sind Langzeitautonomie (Monate ohne Wartung), Reproduzierbarkeit zu niedrigen Kosten und vollständige Open-Source-Verfügbarkeit.

Sensoren (I2C-Bus):

- **SHT45** (Sensirion): Temperatur $\pm 0,1^\circ\text{C}$, Feuchte $\pm 1,0\%$ rH
- **BMP390** (Bosch): Luftdruck $\pm 0,5\text{ hPa}$
- **LTR390** (Lite-On): UV-Index (UVA + ALS)
- **SCD41** (Sensirion): CO_2 -Konzentration $\pm 40\text{ ppm}$ (photoacoustisch, Single-Shot)
- **SPS30** (Sensirion): Feinstaub $\text{PM}_{1.0/2.5/4.0/10.0}$ und Zahlkonzentration, 5 V
- **MAX17048** (Maxim): LiPo-Fuel-Gauge über I2C

Anwendungsszenarien: Landwirtschaftliche Feldmessungen, wissenschaftliche Langzeitstudien, partizipative Luftqualitätsnetzwerke, Frühwarnsysteme für Naturkatastrophen. Die offene I2C-Architektur erlaubt beliebige Sensor-Erweiterungen. Reproduktionskosten: Minimalvariante $< 30\text{ €}$, vollbestückt 120–150 €.

2. Hardware

Prototyp

Die gesamte Software-Entwicklung startete auf dem **Seeed Studio Wio-E5 Mini** (STM32WL55JC): Sensoren als Breakout-Boards auf Steckbrett, alle sechs Sensoren über denselben I2C-Bus (SDA: PA11, SCL: PA12, $4,7\text{ k}\Omega$ Pull-ups) wie im späteren Custom PCB. Erst nachdem die Software vollständig funktionierte und das PCB-Design abgeschlossen war, wurde das Custom PCB bei JLCPCB bestellt. Geflasht wurden sowohl das Wio-E5 Mini als auch das Custom PCB über einen **ST-Link V2**, angeschlossen via RST, CLK, DIO und GND.

Sensor	Hersteller	Messgrößen	I2C-Addr.	Breakout
SHT45	Sensirion	Temperatur, Feuchte	0x44	Adafruit
BMP390	Bosch	Luftdruck	0x77	Adafruit
LTR390	Lite-On	UV, Umgebungslicht	0x53	Adafruit
SCD41	Sensirion	CO_2	0x62	Seeed Studio
SPS30	Sensirion	Feinstaub (PM/NC)	0x69	Sensirion
MAX17048	Maxim	Akkuspannung	0x36	Adafruit

Tabelle 1: Sensoren im Prototyp

Custom PCB

Das Herzstück bildet der **STM32WLE5CCU6**. Ein SoC mit ARM Cortex-M4 (48 MHz) und integriertem Sub-GHz-LoRa-Transceiver. Das selbst entworfene **4-Lagen-PCB** (EasyEDA Pro) wurde vollständig neu entwickelt, da kein geeignetes Open-Source-Design für diesen Chip verfügbar war.

Lagenaufbau: Top und Bottom tragen Signal- und Versorgungsleitungen; Inner 1 ist dedizierte GND-Lage (durchgängiges Kupferpour); Inner 2 wird ebenfalls als Signallage genutzt. Die durchgehende GND-Innenlage ermöglicht kurze Rückstrompfade und reduziert Gleichtaktstörungen auf den I2C- und Steuerleitungen.

RF-Frontend: Balun BALFHB-WL-02D3 und TX/RX-Schalter BGS12SN6 (GPIO PC13). Leiterbahnen im RF-Bereich sind auf 50 Ω Controlled Impedance ausgelegt (868 MHz). Rund um das RF-Signal-Feld (Balun, Schalter, Entkopplungskondensatoren, SMA-Pfad) wurde ein dichter GND-Via-Zaun gesetzt, der das RF-Feld nach unten an die GND-Lage anbindet und seitliche Abstrahlung in andere Boardbereiche unterdrückt. Antenne: externe **SMA-Buchse**.

Energieversorgung: Zwei **TPS63900** Buck-Boost-Wandler ($< 0,5 \mu A$ Quiescent, $> 95 \%$ Effizienz) liefern 3,3 V (Primär, immer an) und 5 V (Sekundär, software-gesteuert via GPIO für SPS30). Solar-MPPT-Laden übernimmt der **BQ25185**.

Anschlüsse:

- **Akku:** 2-poliger JST-Stecker sowie parallel ein 2-poliger Schraubklemmenblock.
- **Solar:** 2-poliger Schraubklemmenblock (neben Akku-Klemme).
- **Stevensonscreen-Kabel:** 6-poliger Schraubklemmenblock für das 6-adrige Verbindungskabel zum Stevenson-Screen-Gehäuse (I2C, Versorgung, Solarpanel-Leitung).
- **SWD-Debugger:** Pinheader (NRST, SWDCLK, SWDIO, GND) für ST-Link V2.

Testpads sind für kritische Signale (Versorgungsspannungen, I2C-Bus, CE-Pin) vorgesehen, um im Fehlerfall mit Logikanalysator oder Multimeter direkt messen zu können. Gefertigt und bestückt bei JLCPCB (5 Platinen, ca. 200 € inkl. Versand).

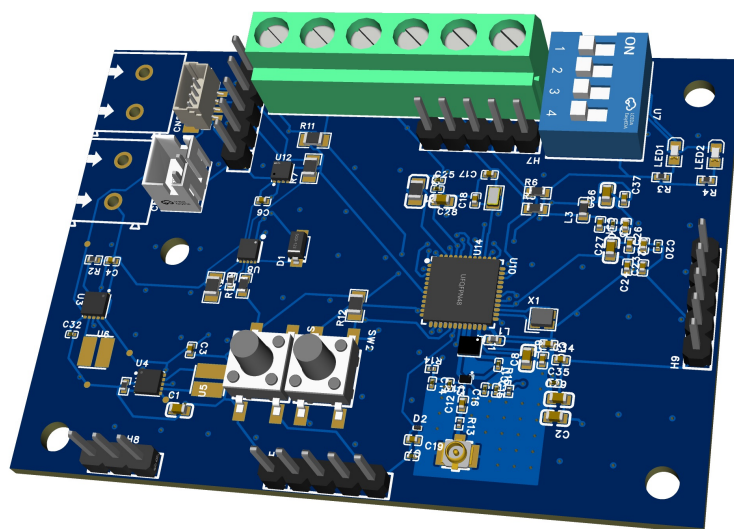


Abbildung 1: 3D-Render des Custom PCB (EasyEDA Pro)

Gehäuse: Stevenson Screen

Ein selbst in Fusion360 entworfenes, mehrlagiges Lamellen-Gehäuse aus weißem **ASA-Kunststoff** (UV-beständig, witterungsfest) schützt die Sensoren vor Direktsonne und Regen bei freier Luftzirkulation. Mehrere 3D-Druckdurchläufe für Passgenauigkeit nötig. Der **LTR390-UV-Sensor** ist hinter einer Polystyrol-Abdeckung (ca. 2 mm) verbaut; ein empirischer Korrekturfaktor $17/13 \approx 1,31$ wird als Ganzzahl-Bruchrechnung in der Firmware angewandt.



Abbildung 2: Montierte HydroNodeStation im Außeneinsatz

3. Firmware und Software

Build-System und Entwicklungsinfrastruktur

CMake + ARM GCC (`arm-none-eabi-gcc`). Das Build-System wurde von Anfang an zweigleisig gefahren: Das **Debug-Target** (Wio-E5 Mini, `-00 -g3`) existierte von Beginn der Prototyping-Phase an; das **Release-Target** (Custom PCB, STM32WLE5CCU6, `-0s`) wurde parallel entwickelt und angepasst, sobald das PCB verfügbar war. Beide Targets teilen die gesamte Applikationslogik; nur Hardware-spezifische Pin-Mappings und MCU-Variante unterscheiden sich. GitHub Actions prüft beide Targets bei jedem Push. Geflasht wurde per **ST-Link V2** über SWD (RST, CLK, DIO, GND) mit JetBrains CLion + OpenOCD. Eine eigene **Remote-Entwicklungsinfrastruktur** (dedizierter Server, OpenOCD als Systemdienst, serieller Konsolen-Dienst via SW-UART auf PA6 mit 115200 Baud) ermöglicht Flashen, Debuggen und Live-Monitoring von beliebigen Standorten.

Firmware-Architektur

Boot-Sequenz: HAL-Init → SystemClock (48 MHz MSI, LSE 32,768 kHz für RTC) → GPIO → I2C2 → LoRaWAN-Init → Sensor-Init → Hauptschleife (`MX_LoRaWAN_Process()`).

Hauptschleife / Low-Power: Nach jedem `smtc_modem_run_engine()`-Aufruf tritt das System in **STOP2** ein (RTC-Alarm + EXTI aktiv, alle anderen Peripherien aus). Weckung erfolgt per RTC-Alarm zum nächsten Sendezeitpunkt.

60-Minuten-Superzyklus: Ein Tx-Zähler (1–30, Basis-Intervall 2 min) steuert die Sensor-Aktivierung. Temperatur/Feuchte/Druck/UV/Akku werden bei *jedem* Uplink gemessen (schnell, energiearm). CO₂ (SCD41) nur alle 20 min (jeder 10. Uplink), Feinstaub (SPS30) nur alle 30 min (jeder 15. Uplink). **Pre-Measurement-Timer** starten SCD41 (5,5 s vorher) und SPS30 (16,5 s vorher), damit der MCU während der Messung in STOP2 schläft.

Payload-Format (Big-Endian, 2 Byte pro Feld):

fPort	Felder	Skalierung	Bytes
2	Temp, Feuchte, Druck, Akku, UV	i16/u16, 0,01/0,01/0,1/1/0,01	10
3	wie fPort 2 + CO ₂	u16, 1 ppm	12
4	wie fPort 3 + PM1.0/2.5/4.0/10.0, NC, TypSize	u16 div.	32

Tabelle 2: Uplink-Payload-Format (alle Felder Big-Endian)

Batterie-adaptives Verhalten:

Akkuspannung	Intervall	Verhalten
≥ 3400 mV	2 min	Vollbetrieb
< 3400 mV	4 min	Intervall ×2
< 3000 mV	12 min	Intervall ×6, nur fPort 2

Downlink-Befehle: 0x10 HLL – TX-Intervall setzen (2-Byte Big-Endian Sekunden, 30–3600 s, flash-persistent, z. B. zum Testen ohne Neu-Flashen); 0x11 – SPS30 Fan Cleaning; 0xFF – Software Reset. NVM-Persistenz (Session-Keys, Frame-Counter) im letzten Flash-Sektor: kein Re-Join nach Neustart nötig. Device-EUI wird automatisch aus der 96-Bit-UID des STM32 abgeleitet.

Sensor-Treiber (`sys_sensors.c`): `EnvSensors_Init()` – tolerant bei fehlenden Sensoren. `EnvSensors_Read(flags)` – automatischer I2C-Bus-Recovery bei Fehlern (relevant bei SPS30-Gebläse-Spikes). SPS30 alle 120 h automatische Fan-Reinigung (wenn Akku > 4120 mV).

4. Inbetriebnahme

Schritt 1: Hardware beschaffen

Zwei Optionen:

Option	Details
Custom PCB (empfohlen)	Schaltplan, BOM und Gerber-Dateien im Repository (<code>hardware/ordered_pcb/</code>). Bestellung + maschinelle Bestückung bei JLCPCB, 5 Stück ca. 200 € inkl. Versand. Lieferzeit ca. 2–3 Wochen. Flashen via ST-Link V2 (RST, CLK, DIO, GND).
Seeed Studio Wio-E5 Mini (Prototyping)	Fertig verfügbares Entwicklungsboard mit STM32WL55JC + LoRa-Transceiver und Stiftleisten. Kein Lötkolben nötig. Sensoren als Breakout-Boards über Jumper-Kabel anschließen (alle sechs Sensoren unterstützt; gleicher I2C-Bus wie Custom PCB). Flashen ebenfalls via ST-Link V2 (RST, CLK, DIO, GND).

Für das Wio-E5 Mini werden zusätzlich benötigt: Breakout-Boards für SHT45, BMP390, LTR390, SCD41, SPS30 und MAX17048 (alle über I2C), Jumperwires sowie ein Step-Up-Konverter auf 5 V für den SPS30.

Schritt 2: Build-Voraussetzungen installieren

Werkzeug	Version	Zweck
arm-none-eabi-gcc	≥ 12	ARM GCC Toolchain
cmake	≥ 3.22	Build-System
ninja	beliebig	CMake-Generator
ST-Link V2	Hardware	Flashen via SWD
openocd	beliebig	Flash-Utility / GDB-Server

macOS: `brew install arm-none-eabi-gcc cmake ninja`

Ubuntu: `sudo apt install gcc-arm-none-eabi cmake ninja-build`

Schritt 3: Repository klonen und Firmware initial bauen

```
git clone https://github.com/TexhFexLabs/HydroNodeStation.git
cd hydroNodeStation/stm32_node
```

Custom PCB (Release):

```
cmake -B build/Release \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_TOOLCHAIN_FILE=cmake/gcc-arm-none-eabi.cmake
cmake --build build/Release
```

Wio-E5 Mini (Debug):

```
cmake -B build/Debug \
  -DCMAKE_BUILD_TYPE=Debug \
  -DCMAKE_TOOLCHAIN_FILE=cmake/gcc-arm-none-eabi.cmake
cmake --build build/Debug
```

Die Datei `se-identity.h` bleibt zunächst unverändert: `LORAWAN_DEVICE_EUI` auf `00,00,...,00` lassen. Die Firmware leitet die Device-EUI automatisch aus der Hardware-UID des STM32 ab.

Schritt 4: Erstmaliges Flashen und Device-EUI auslesen

Beide Boards werden identisch über einen **ST-Link V2** geflasht. ST-Link V2 an die SWD-Pins anschließen (RST, CLK, DIO, GND; Pinbelegung siehe Schaltplan). Anschließend mit **STM32CubeProgrammer** (GUI) oder **JetBrains CLion + OpenOCD** die erzeugte `.elf`-Datei flashen:

- Custom PCB: `build/Release/LoRaWAN_End_Node_LBM.elf`
- Wio-E5 Mini: `build/Debug/LoRaWAN_End_Node_LBM.elf`

Nach dem Flashen: LED blinkt 10 Sekunden. Anschließend USB-UART-Adapter an **PA6** (115200 Baud) anschließen. Die Firmware gibt beim Boot die abgeleitete **Device-EUI** in der seriellen Konsole aus. Diese notieren.

Schritt 5: Gerät beim Netzwerk-Server registrieren

Mit der ausgelesenen Device-EUI ein neues OTAA-Gerät anlegen (LoRaWAN 1.0.4, EU868). Unterstützte Netzwerk-Server: **Helium IoT**, **ChirpStack v4**, **The Things Network (TTN)**.

Der Netzwerk-Server generiert dabei **Join EUI** (App EUI) und **App Key**. Diese beiden Werte werden im nächsten Schritt in den Code eingetragen.

Schritt 6: LoRaWAN-Schlüssel eintragen, neu bauen und flashen

Datei: `stm32_node/LoRaWAN/App/se-identity.h`

```
#define LORAWAN_DEVICE_EUI    00,00,00,00,00,00,00,00 // immer auf 00 lassen
#define LORAWAN_JOIN_EUI     AA,BB,CC,DD,EE,FF,00,11 // vom Netzwerk-Server
#define LORAWAN_APP_KEY      AA,BB,...,99             // vom Netzwerk-Server
#define LORAWAN_GEN_APP_KEY  AA,BB,...,99             // gleich wie APP_KEY
```

Schlüssel nicht committen: `se-identity.h` in `.git/info/exclude` eintragen. Danach Firmware neu bauen (`cmake -build build/Release`) und erneut flashen wie in Schritt 4. Die Station startet dann den OTAA-Join mit den echten Credentials.

Schritt 7: LoRaWAN-Anbindung über HTTP-Webhook

Der Netzwerk-Server (Helium IoT, ChirpStack v4 oder TTN) sendet jeden Uplink per **HTTP-Webhook direkt** an das HydroNode-Backend – kein AWS SNS o. ä. dazwischen. Die konkrete Webhook-URL stellt HydroNode in Schritt 8 bereit; sie wird im Netzwerk-Server unter *Integrations* → *HTTP/Webhook* hinterlegt. Das Backend bestätigt eingehende Uplinks automatisch.

Schritt 8: Station in HydroNode aktivieren

1. HydroNode-App öffnen oder `https://hydronode.texhexlabs.de` aufrufen.
2. Registrierung: erster Sensor-Slot kostenlos enthalten.

3. Sensor auswählen → **Einstellungen** → **LoRaWAN-Binding**.
4. **Device-EUI** (aus Schritt 4) eintragen und auf **Save** drücken.
5. Nach dem Speichern zeigt HydroNode einen **Webhook-URL** an. Diesen URL im Netzwerk-Server (TTN, ChirpStack oder Helium) unter **Integrations** → **Webhook** eintragen.
6. Ab diesem Zeitpunkt sendet der Netzwerk-Server jeden eingehenden Uplink automatisch an das HydroNode-Backend; keine weitere Konfiguration nötig.

Das HydroNode-Backend verarbeitet die Uplinks vollautomatisch: Payload-Dekodierung anhand des **fPort**, Datenbankablage und KI-Anomalieerkennung laufen ohne weiteren Setup. Messwerte erscheinen ab dem nächsten Uplink im Dashboard und in der App.

Schritt 9: Betrieb verifizieren und Debug-Ausgabe

Debug-Ausgabe (SW-UART): USB-UART-Adapter an **PA6**, 115200 Baud. Logging im normalen LoRaWAN-Betrieb: `APP_LOG_ENABLED 1` in `Core/Inc/sys_conf.h`.

Debug-Profil-Modus: PB4 beim Einschalten HIGH → startet ohne LoRaWAN, liest alle Sensoren im 8s-Takt, Rohwerte über PA6. Nützlich für Sensor-Verifikation und Kalibrierung.

Wichtige konfigurierbare Parameter (in `lora_app.h`):

- `APP_TX_DUTYCYCLE` (Standard 300 000 ms = 5 min): Sendeintervall
- `LOW_BAT_TX_DUTYCYCLE` (600 000 ms): Intervall bei < 3400 mV
- `CRITICAL_BAT_TX_DUTYCYCLE` (3 600 000 ms): Intervall bei < 3000 mV
- `SCD41_PREMEASUREMENT_MS` (5500): Vorlaufzeit CO₂ (min. 5000)
- `SPS30_PREMEASUREMENT_MS` (16500): Vorlaufzeit Feinstaub (min. 16000)

RF-Region: `ACTIVE_REGION` in `LoRaWAN/Target/lorawan_conf.h` (EU868, US915, AS923 etc.).

5. Energieeffizienz

Komponente	Zustand	Strom	Quelle
STM32WLE5	STOP2 + RTC	$1,07 \mu\text{A}$	DS13105
STM32WLE5	Aktiv (48 MHz)	$\approx 5 \text{ mA}$	Datenblatt
STM32WLE5	LoRa TX @14 dBm	$\approx 22 \text{ mA}$	Datenblatt
SCD41	Power-down	$\approx 0 \mu\text{A}$	Sensirion
SHT45	Deep Sleep	$0,1 \mu\text{A}$	Datenblatt
BMP390	Sleep	$2,0 \mu\text{A}$	Datenblatt
MAX17048	Sleep	$1,5 \mu\text{A}$	Datenblatt
TPS63900	Idle (je Regler)	$< 0,5 \mu\text{A}$	Datenblatt

Tabelle 3: Komponentenstromverbrauch (Datenblattwerte)

Theoretischer STOP2-Systemstrom: $I_{\text{sleep,theo}} = 1,07 + 0,10 + 2,00 + 1,50 + 2 \times 0,5 \approx 5,67 \mu\text{A}$.

Gemessene Tagesmittel-Baseline: ca. **1,1 mA**. Mehrere Größenordnungen über Theorie, vermutlich durch SCD41-IR-Leckstrom, BQ25185-Quiescent und PCB-Leckströme. Zum Vergleich: Steckbrett-Prototyp (Wio-E5): ca. 3 mA (Onboard-LEDs, CP2102, Pololu-Step-Up 400 μA).

Tagesverbrauch (PPK2-Messung): Aus der Baseline und den Mehrladungen der einzelnen Sendeereignisse ergibt sich der mittlere Ladungsbedarf pro Tag.

Quelle	Intervall	Mehrladung/Ereignis	Ereignisse/Tag	mC/Tag
Baseline ($1,1 \text{ mA} \times 86400 \text{ s}$)	—	—	—	95.040
LoRa TX ($+0,7 \text{ mA} \times 10 \text{ s}$)	3 min	7 mC	480	3.360
SCD41 CO ₂ ($+77 \text{ mC}$)	15 min	77 mC	96	7.392
SPS30 ($55 \text{ mA} \times 30 \text{ s} - \text{idle}$)	30 min	$\approx 1.617 \text{ mC}$	48	77.616
Gesamt				≈ 183.408

Tabelle 4: Tagesverbrauch nach Quelle (PPK2-Messung, Custom PCB)

Gesamtladung pro Tag: $\approx 183.408 \text{ mC} / 3600 = \approx 50,9 \text{ mAh/Tag}$. Daraus folgt ein mittlerer Systemstrom $I_{\text{avg}} = 50,9 \text{ mAh} / 24 \text{ h} \approx 2,12 \text{ mA}$ – deutlich über dem reinen STOP2-Strom, da die Sendeereignisse (v.a. SPS30-Gebläse) den Mittelwert dominieren.

Akku	I_{avg}	Autonomie ohne Solar
1200 mAh	$\approx 2,12 \text{ mA}$	$\approx 24 \text{ Tage}$

Tabelle 5: Batterielaufzeit (1200 mAh/50,9 mAh/Tag, PPK2-Messung, Custom PCB)

6. Tests und Validierung

Prototyp: Alle sechs Sensoren auf Plausibilität und I2C-Korrektheit verifiziert. LoRaWAN-OTAA-Join und Uplinks auf fPort 2/3/4 stabil. Feldtest: > 10 km Reichweite nachgewiesen. NVM-Persistenz durch Neustarts bestätigt. I2C-Bus-Recovery durch absichtlich induzierte Fehler verifiziert. Downlinks 0x11 und 0xFF funktional.

Custom PCB: TPS63900-Fehler identifiziert und mit Breakout-Workaround behoben. Alle fünf Kernsensoren verifiziert. LoRaWAN-Uplinks fließen per HTTP-Webhook direkt ins HydroNode-Backend. PPK2-Strommessung durchgeführt. Station seit Ende Mai 2026 im Außen-Feldtest.

Live-Dashboard: <https://hydronode.texhflexlabs.de/stations/public/004dd7e7-5c27-4de0-bff5-24116f721658>

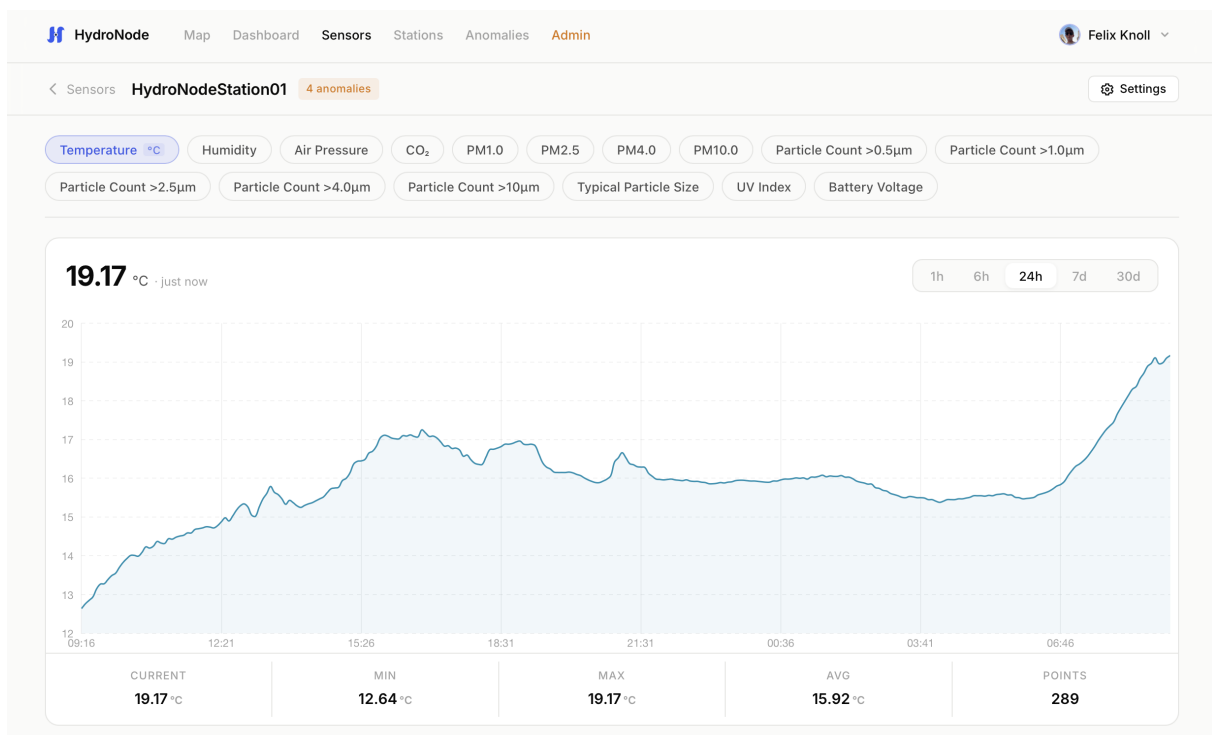


Abbildung 3: Live-Dashboard der HydroNodeStation

Noch ausstehend: Solar-Autarkie-Verifikation (BQ25185) unter verschiedenen Lichtbedingungen; Langzeit-Batterie-Laufzeitmessung.

7. HydroNodeNetzwerk

Das HydroNodeNetzwerk verbindet Sensorstationen mit Backend und iOS-App. Es unterstützt zwei Übertragungspfade:

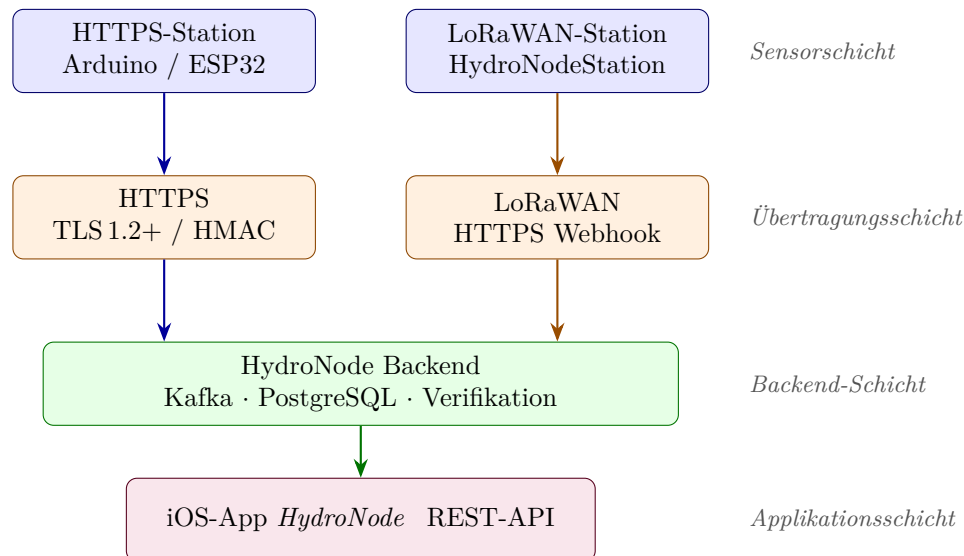


Abbildung 4: Systemarchitektur des HydroNodeNetzwerks

HTTPS-Pfad: Stationen signieren JSON-Payloads mit HMAC-SHA256 (Header X-Signature, Replay-Schutz via X-Timestamp ± 20 min). Backend publiziert in Kafka-Topic `hydronode.ingress.sensor` → `DeviceKeyVerifier` → `hydronode.verified.sensor` → `SensorValueProcessor` → PostgreSQL.

LoRaWAN-Pfad: Station → Netzwerk-Server (Helium IoT / ChirpStack v4 / TTN) → HTTP-Webhook → HydroNode-Backend. Backend ermittelt Sensor via `SensorLookupCache` (DevEUI), dekodiert Payload per `LoRaFPortConfig` (Big-Endian, kanalweise Skalierung) und publiziert direkt nach `hydronode.verified.sensor`.

Backend: Quarkus-3-Anwendung. In-Process-Caches (`DeviceKeyCache`, `SensorLookupCache`, `FPortConfigCache`, TTL 90 min) reduzieren DB-Last. Kafka entkoppelt Empfang und Verarbeitung.

KI-Analyse (Amazon Bedrock Nova Micro): Alle 15 min automatische Anomalieerkennung (SPIKE, DROP, GRADUAL_DRIFT, THRESHOLD_BREACH) mit 48 h Historienkontext. Alle 6 h Routineanalyse aller aktiven Sensoren. Antwort enthält Erklärung, Konfidenz, Alert-Flag und Handlungsempfehlung. Outdoor- und Indoor-Sensoren erhalten unterschiedliche System-Prompts.

8. Open-Source-Veröffentlichung

Lizenzen: Hardware (Schaltplan, PCB, CAD-Gehäuse) unter **CC BY-NC-SA 4.0**; Firmware (stm32_node) unter **BSD-3-Clause**.

Veröffentlichung: <https://github.com/TeXhFexLabs/HydroNodeStation> (Schaltplan-PDF, PCB-Renders, BOM, vollständige CMake-Firmware, Inbetriebnahme-Anleitung und 3MF-Dateien des Gehäuses).

OSHWLab Start 2026 Contest: Projekt angemeldet – ein Wettbewerb, bei dem Open-Hardware-Erfindungen und Prototyp-PCBs eingereicht und ausgezeichnet werden.

9. Stundenaufwand

Projektplan: 20 Features, **160 Stunden** geplant. Tatsächlich laut Arbeitstagebuch: **170 Stunden**.

Wichtigste Abweichungen: PCB-Design mit ca. 50 h weit über den geplanten 9 h (+41 h). 3D-Gehäuse mit ca. 12 h ungeplanter Zusatzaufwand. KI-Evaluation mit –14 h deutlich unter Plan (PCB-Inbetriebnahme hatte Priorität). Solar-Lade-Bug-Fix und Projekt-Cleanup nach Abschluss: +8 h. Einsparungen in anderen Bereichen gleichen den Mehraufwand teilweise aus; Gesamtabweichung +10 h.

Detaillierter Soll/Ist-Vergleich nach Features und Bereichen sowie vollständiges Arbeitstagebuch: siehe Anhang A.

10. KI-Verwendung

Im Projektverlauf wurde **Claude Code** (Anthropic) als KI-Werkzeug für Dokumentation, Code-Analyse und Problemlösung eingesetzt. Testweise wurde außerdem **Gemini CLI** (Google) zum Vergleich genutzt. Empfehlung: Token-Verbrauch aktiv überwachen, Konversationen thematisch bündeln und KI-Werkzeuge gezielt einsetzen, da Kosten bei langen Kontexten schnell steigen.

11. Abweichungen, Probleme und Lösungen

Architekturelle Abweichungen vom Projektplan

SW-UART statt Hardware-USART für Debug-Ausgabe. Im Projektplan war USART1 als Debug-Schnittstelle vorgesehen. Im ersten PCB-Revisionsstand lag USART1 auf einem falschen Pin; die Debug-Ausgabe wurde daher auf eine **software-emulierte UART** (`sw_uart.c`) auf Pin PA6 umgestellt. Im überarbeiteten PCB-Design ist der Fehler behoben.

Dualer Build-Target (Prototyp + Custom PCB). Im Plan war ein einzelnes Firmware-Target vorgesehen. Tatsächlich existierten zwei Targets von Beginn an: das **Debug-Target** (Wio-E5 Mini, STM32WL55, -00 -g3) für die gesamte Prototyping-Phase und das **Release-Target** (STM32WLE5CCU6, -0s) als Anpassung für das Custom PCB. Applikationslogik identisch; nur Pin-Mappings und MCU-Variante unterscheiden sich.

Pre-Measurement-Timer-Architektur. Der Projektplan sah einen einfachen Messzyklus (Aufwachen → Messen → Senden → Schlafen) vor. Tatsächlich wurde eine **asynchrone Pre-Measurement-Logik** eingeführt: SCD41 und SPS30 werden durch separate Timer *vor* dem Sendezeitpunkt geweckt (SCD41: 5,5 s, SPS30: 16,5 s), damit der MCU während der Messzeit in STOP2 schläft und aktives Warten entfällt.

60-Minuten-Superzyklus mit sensorspezifischen Häufigkeiten. Im Plan war gleichmäßige Übertragung aller Sensorwerte bei jedem Uplink vorgesehen. Tatsächlich wurde ein **Tx-Zähler-basiertes Superzyklus-Modell** eingeführt: Temperatur/Feuchte/Druck/UV bei jedem Uplink, CO₂ alle 20 min, Feinstaub (SPS30) alle 30 min. Der SPS30-Gebläsebetrieb ist energetisch dominant; seltene Aktivierung reduziert den mittleren Verbrauch erheblich.

Probleme und Lösungen

PCB-Aufwand deutlich höher als geplant. 9 h geplant, ca. 56 h tatsächlich aufgewendet. Kein geeignetes Open-Source-Design für den STM32WLE5JC verfügbar; vollständige Neuentwicklung in EasyEDA Pro mit mehreren Iterationen für RF-Design, Antennenmatching und Komponentenplatzierung. Nach Lieferung zusätzliche Bugs (u. a. TPS63900, SW-UART). Mehraufwand durch Einsparungen in anderen Bereichen und Verzicht auf Lautstärke-Sensor teilweise ausgeglichen; Gesamtabweichung +10 h.

Fehler im TPS63900-Spannungsregler. Nach Lieferung der JLCPCB-Platinen funktionierte der primäre TPS63900 (3,3-V) nicht; Designfehler im Schaltplan (nach Absprache mit Experten identifiziert). **Lösung:** Chip von LiPo-Plus und 3,3-V-Netz getrennt; identischer Chip auf Breakout-Board montiert und an PCB-Header angesteckt. System läuft vollständig funktionsfähig mit identischer Effizienz. Korrigiertes PCB-Design für zukünftige Revisionen bereit.

BQ25185-Solarlade-Bug: 6-h-Timeout. Im Feldtest: Laden startete täglich ca. 07:00 Uhr und stoppte um ca. 13:00 Uhr, obwohl die Solarzelle noch Energie lieferte. Ursache: interner 6-Stunden-Sicherheits-Countdown des BQ25185; bei dauerhaft anliegendem CE-Pin bricht der Chip nach 6 h das Laden ab. **Lösung (17.06.2026):** STM32-GPIO direkt an CE-Pin des BQ25185 angelötet; bei jeder Messung (alle 5 min) wird CE für ca. 200 ms auf HIGH gezogen und der interne Countdown resettet, ohne den Ladevorgang zu unterbrechen.

Sensor-Wechsel gegenüber Projektplan.

- BME280 → **SHT45** + **BMP390**: höhere Genauigkeit ($\pm 0,1\text{ °C}$ / $\pm 0,5\text{ hPa}$), getrennte Sleep-Modi.
- VEML6075 → **LTR390**: breiterer UV-Messbereich, I2C-Ausgang statt analog, direkter UVI-Output.
- **MAX4466-Lautstärke-Sensor entfallen**: Signalverarbeitung im Zeitrahmen nicht sauber realisierbar; alle Kern-Umweltparameter dennoch vollständig erfasst.

3D-Gehäuse: ungeplanter Mehraufwand. Kein Gehäuse-Design im Projektplan vorgesehen; ca. 12 h tatsächlich aufgewendet. Für korrekte Außenmessungen zwingend erforderlich: Stevenson-Screen-Design (überlagerte Lamellen, kein direkter Sonnenkontakt, freie Luftzirkulation). Mehrere Druckdurchläufe, Material weißes ASA (UV-beständig, witterungsfest).

UV-Sensorschutz: Kunststofffenster mit Korrekturfaktor. LTR390 muss vor Schmutz und Niederschlag geschützt werden, darf aber UV-Licht empfangen. UV-transparentes Quarzglas (50 €) wirtschaftlich nicht vertretbar. **Lösung:** Polystyrol-Fenster (ca. 2 mm). Empirischer Korrekturfaktor aus Vergleichsmessungen:

$$\frac{\text{UVI}_{\text{ohne}}}{\text{UVI}_{\text{mit}}} = \frac{6,8}{5,2} = \frac{17}{13} \approx 1,31$$

In Firmware als Ganzzahl-Bruchrechnung implementiert (`uvi_raw × 17 / 13`).

KI-Evaluation nicht abgeschlossen. 16 h geplant, ca. 2 h investiert. PCB-Inbetriebnahme hatte Priorität. KI-Anomalieerkennung läuft bereits produktiv im HydroNode-Backend (Amazon Bedrock Nova Micro, alle 15 min); formale Evaluation folgt nach Hardware-Stabilisierung.

Versionsverwaltung und Arbeitsablauf. Branches enthielten häufig mehrere inhaltlich unterschiedliche Änderungen, was Nachvollziehbarkeit und Review erschwerte. Für zukünftige Arbeiten: konsequent thematisch getrennte Branches, je Änderung ein eigenes Issue und Pull Request.

Ergebnisse

- Funktionsfähiges Custom PCB (4-Lagen, STM32WLE5CCU6) im Außeneinsatz seit Ende Mai 2026.
- Alle fünf Kernsensoren (SHT45, BMP390, LTR390, SPS30, SCD41) validiert, stabile Feldtest-Daten.
- LoRaWAN-Pipeline (Helium → HTTPS Webhook → HydroNode Backend) aktiv.
- STOP2-Ruhestrom ca. 900 µA (gemessen, PPK2, Custom PCB).
- Live-Dashboard und iOS-App operativ. Open-Source-Veröffentlichung auf GitHub abgeschlossen.

Offene Punkte: Solar-Autarkie-Verifikation; Langzeit-Batterie-Laufzeitmessung; KI-Evaluation; permanente Feldtest-Aufstellung an der OTH Regensburg.

12. Fazit

Das Projekt hat sein Kernziel erreicht: Eine selbst entworfene, solar-betriebene Umweltmessstation läuft seit Ende Mai 2026 im Außeneinsatz und überträgt zuverlässig Messdaten über LoRaWAN. Alle fünf Kernsensoren liefern stabile Felddaten, die Pipeline bis ins Backend funktioniert end-to-end.

Der größte Lerneffekt war das PCB-Design. Was im Projektplan mit 9 h veranschlagt war, hat am Ende rund 56 h gekostet. Das liegt nicht daran, dass der Plan schlecht war, sondern daran, dass RF-Design, 4-Lagen-Stackup und LoRa-Antennenpfad Themen sind, die man erst wirklich versteht, wenn man sie selbst debuggt. Die Bugs nach der ersten Lieferung (TPS63900, SW-UART, und zuletzt der BQ25185-Solar-Timeout) waren frustrierend, aber gleichzeitig die lehrreichsten Momente des Projekts. Jeder dieser Fehler hat ein konkretes Verständnis hinterlassen, das kein Lehrbuch so direkt vermitteln kann.

Was mich am meisten überrascht hat: Das System läuft tatsächlich. Es steht draußen, sendet Daten, das Dashboard zeigt Live-Werte. Das klingt selbstverständlich, ist es aber nicht. Zwischen Prototyp auf dem Steckbrett und funktionierendem Custom PCB im Stevenson-Screen-Gehäuse liegt ein langer Weg, den man erst einmal gehen muss.

Offen bleibt die Langzeit-Solar-Autarkie-Verifikation. Das System ist dafür ausgelegt, aber belastbare Daten über mehrere Wochen stehen noch aus. Das ist der nächste logische Schritt.

A. Stundenaufwand: Detailtabellen

Soll/Ist-Vergleich nach Features

#	Feature	Geplant	Ist	Status
1	Hardware-Auswahl & Beschaffung	8 h	7 h	Fertig
2	Schaltplan & Elektronik-Aufbau	9 h	4 h	Fertig
3	Sensor: Temperatur / Feuchte / Druck	2 h	3 h	Fertig
4	Sensor: UV-Index (LTR390)	2 h	2 h	Fertig
5	Sensor: Feinstaub SPS30	4 h	3 h	Fertig
6	Sensor: CO ₂ SCD41	4 h	3 h	Fertig
7	Sensor: Lautstärke (MAX4466)	3 h	0 h	Weggelassen
8	Standalone-Tests	8 h	3 h	Fertig
9	PCB-Design, Bestellung & Inbetriebnahme	9 h	~56 h	Fertig
10	Autarke Stromversorgung Solar	13 h	~7 h	In Arbeit
11	Deep-Sleep & Power-Management	7 h	~10 h	Fertig
12	LoRaWAN-Firmware	12 h	~11 h	Fertig
13	Helium & Decoder	5 h	~3 h	Fertig
14	Backend: Pipeline zu HydroNode	8 h	~3 h	Fertig
15	Website Echtzeit-Dashboard	14 h	10 h	Fertig
16	App: Verlaufsdiagramme	6 h	~6 h	Fertig
17	App: Stationsverwaltung	4 h	~6 h	Fertig
18	KI-Eval: Datensatz & Modellauswahl	8 h	~2 h	In Arbeit
19	KI-Eval: Testing & Benchmarking	8 h	0 h	Geplant
20	Feldtest & Abschlussdoku	16 h	~7 h	In Arbeit
–	3D-Gehäuse (nicht geplant)	0 h	~12 h	Fertig
Gesamt		160 h	170 h	

Soll/Ist-Vergleich nach Bereichen

Bereich	Geplant (h)	Ist (h, est.)	Differenz
Hardware, Beschaffung, Schaltplan	17	~9	–8
Sensorik (5 von 6 Sensoren)	12	~11	–1
Standalone-Tests	8	~3	–5
PCB-Design, Bestellung, Inbetriebnahme	9	~50	+41
Autarke Stromversorgung (Solar)	13	~7	–6
Firmware (Deep-Sleep, LoRaWAN)	19	~21	+2
Helium-Netzwerk & Backend	13	~6	–7
Website & App	24	~23	–1
KI-Evaluation	16	~2	–14
Feldtest & Dokumentation	16	~10	–6
3D-Gehäuse (nicht geplant)	0	~12	+12
Gesamt (laut Arbeitstagebuch)	160	170	+10

Tabelle 7: Stundenvergleich Soll/Ist nach Bereichen

Arbeitstagebuch

Datum	h	Kategorie	Beschreibung
23.03.2026	5	Hardware	Energieeffiziente Kernkomponenten ausgewählt und bestellt (STM32, Laderegler, Konverter, SCD41).
27.03.2026	4	Software	Testprojekt aufgebaut, Dummy-Werte stabil via LoRaWAN übertragen.
29.03.2026	2	Hardware	Lötarbeiten (STM32, BQ25185), Inbetriebnahme vorbereitet.
30.03.2026	4	Software	Erste Firmware per ST-Link geflasht, LoRaWAN-Parameter konfiguriert.
30.03.2026	2	Software	Versandzyklen und Low-Power-Ablauf im LoRaWAN-Sendeprozess getestet.
31.03.2026	2	Hardware	Weitere Sensorik (SHT45, BMP390, LTR390) ausgewählt und nachbestellt.
31.03.2026	1	Hardware	Sendetests mit eigenem und externem Gateway, verschiedene Antennen.
01.04.2026	1	Hardware	LoRa-Funktionstest im Gebäude K104.
01.04.2026	2	Software	Projektstruktur überarbeitet, SCD41 integriert, Build via Makefiles verbessert.
03.04.2026	2	Planung	Mindestanforderungen für Batterie- und Solarversorgung rechnerisch ausgelegt.
05.04.2026	2	Software	Energieeffizienz verbessert, Batteriemonitor LC709203F integriert.
05.04.2026	2	Software	Feldtests bei schwachen Empfangsbedingungen (> 10 km), GitHub-Actions eingerichtet.
06.04.2026	6	CAD	PCB-Design erstellt, Datenblattabgleich und Leistungsbudget-Berechnungen.
07.04.2026	6	CAD	PCB-Layout detailliert ausgearbeitet und für Integrationsschritte vorbereitet.
10.04.2026	6	Software	Kommunikations- und Applikations-Stack verbessert und stabilisiert.
11.04.2026	3	Planung	Vorbereitung PCB-Design und MCU-Wechsel strukturiert abgeschlossen.
13.04.2026	6	Software	SHT45, BMP390, LTR390, MAX17048 implementiert, eingebunden, verifiziert.
14.04.2026	2	Hardware	Signalstärketests am OTH-Standort, Empfang durch mehrere Stationen.
15.04.2026	6	Software	Remote-Debugging eingerichtet, Pre-Wake-Logik verbessert.
16.04.2026	2	Software	SPS30 in Messzyklus integriert und in Betrieb genommen.
16.04.2026	1	Software	Fan-Cleaning-Abläufe nachgeschärft, zugehörige Fehler behoben.
18.04.2026	7	CAD	PCB- und RF-Design finalisiert, für Fertigung freigegeben.
20.04.2026	5	Software	CMake-Projektstruktur für STM32WLE5CCU6 neu aufgebaut.
Fortsetzung auf nächster Seite			

Datum	h	Kategorie	Beschreibung
21.04.2026	1	Planung	LaTeX-Dokumentationsprojekt initial angelegt.
22.04.2026	1	Software	RX-Aktionen in Kommunikationsablauf integriert.
22.04.2026	1	Software	Fehler im manuellen SPS30-Fan-Cleaning reproduziert und korrigiert.
24.04.2026	2	CAD	PCB-Layout finalisiert, letzte Hardwareanpassungen.
24.04.2026	2	Software	Firmware an finalisiertes PCB angepasst, Regressionen geprüft.
26.04.2026	6	Software	Helium-SNS in HydroNodeNetwork integriert, End-to-End-Datenfluss validiert.
27.04.2026	5	Software	HydroNode-App um Funktionen für HydroNodeStationen erweitert.
27.04.2026	3	Software	Firmware an Netzwerkabläufe angepasst, Station in App eingerichtet.
28.04.2026	2	CAD	PCB finalisiert, Bestellung inkl. Bestückung bei JLCPCB aufgegeben.
29.04.2026	2	Software	Energieanalyse ohne Solarladegerät durchgeführt und dokumentiert.
30.04.2026	1	Software	State-of-Charge-Ausgabe des MAX17048 in Messzyklus integriert.
30.04.2026	2	Software	Troubleshooting SubGHz-Fehler (write CMD error 1) analysiert.
01.05.2026	3	Planung	Studiendokumentation erweitert (Hardware, Software, Energie, Testing).
02.05.2026	6	CAD	3D-Gehäuse in Fusion 360 konstruiert und für 3D-Druck aufbereitet.
07.05.2026	3	Hardware	Langzeittest ohne Solar mit WioE5-Prototyp gestartet.
11.05.2026	4	Hardware	PCB-Fehlersuche: defekter TPS63900 als Ursache identifiziert.
12.05.2026	3	Software	Erste Firmware-Uploads auf Custom PCB, LoRa-Core-Probleme eingegrenzt.
16.05.2026	10	Software	Projektwebsite erstellt, mit Live-Messdaten und technischen Details befüllt.
18.05.2026	2	CAD	Sensorgehäuse für Außenmontage modelliert, ersten 3D-Druckdurchlauf gestartet.
20.05.2026	3	CAD	Zweiten Druckdurchlauf abgeschlossen, Passgenauigkeit geprüft.
22.05.2026	6	Software	iOS-App aktualisiert, Fehler behoben, Webapp-Funktionen erweitert.
26.05.2026	4	Software	PCB-Fehler behoben, Firmware erstmals stabil mit Batterieversorgung.
27.05.2026	1	Hardware	Kleinere Solarzelle auf Eignung für Energieversorgung getestet.
28.05.2026	3	Hardware	Outdoor-Gehäuse montiert, Sensoren eingebaut, Feldtest gestartet.
30.05.2026	1	Software	Erste Feldtest-Messwerte auf Plausibilität geprüft.
Fortsetzung auf nächster Seite			

Datum	h	Kategorie	Beschreibung
06.06.2026	2	Hardware	Plexiglasabdeckung für UV-Sensor gefertigt und eingepasst.
08.06.2026	4	Software	UV-Index-Berechnung refaktoriert, Sensorkabel überarbeitet, Testläufe.
17.06.2026	5	Hardware	Solar-Lade-Bug behoben: BQ25185 beendete Laden nach internem 6-h-Timeout automatisch und startete erst bei erneutem Solar-Null-Übergang neu. Ursache: kein regelmäßiges CE-Signal. Fix: STM32-GPIO an CE-Pin des BQ25185 angelötet; bei jeder Messung wird CE für ca. 200 ms auf HIGH gezogen, was den internen Countdown resettet. Tägliches Lademuster 07:00–13:00 Uhr dadurch behoben.
19.06.2026	3	Software/Doku	Projekt-Cleanup für Abgabe und Open-Source-Veröffentlichung: Codestruktur aufgeräumt, Dokumentation finalisiert, Repository-Metadaten gepflegt.
Gesamt	170		

Tabelle 8: Dokumentierter Stundenaufwand nach Datum (Stand: 19.06.2026)